# Embeddings

## Lesson 3: Embedding Operations

# Agenda

- **Similarity**: cosine similarity — comparing embeddings by angle, not length
- **Dimension reduction**: t-SNE — seeing a cloud of embeddings in 2-D, the crowding problem, and what perplexity actually controls (closing with PCA, the linear alternative)
- **Fast retrieval**: locality-sensitive hashing (LSH) — finding near neighbors.

## Learning Objectives

By the end of this lesson, you'll be able to:

- Compute cosine similarity from vector norms and dot products, and explain when it disagrees with Euclidean distance — and why normalizing resolves the conflict.
- Explain *intuitively* how t-SNE builds a 2-D map from neighborhoods, why the flat map needs a kernel that leaves extra room, and what actually changes when you turn the perplexity and iteration knobs — then say where PCA's linear shadow differs and when to prefer it.
- Derive the random-hyperplane LSH collision probability from first principles, and explain the $K$/$L$ speed–recall trade-off it produces.

## Why this matters

Lessons 1–2 built the vectors. This lesson is about *using* them — the three operations that turn a pile of embeddings into a working system.

- Every search engine, recommender, and duplicate-detector answers "how similar are these two things?" with **cosine similarity**.
- Every "look, the clusters separate!" figure relies on squashing hundreds of dimensions onto a screen — almost always with **t-SNE** — and misreading those pictures is one of the most common mistakes in applied ML.
- At real scale (millions of documents), comparing a query to *everything* is too slow — **LSH** is how production retrieval systems stay fast.

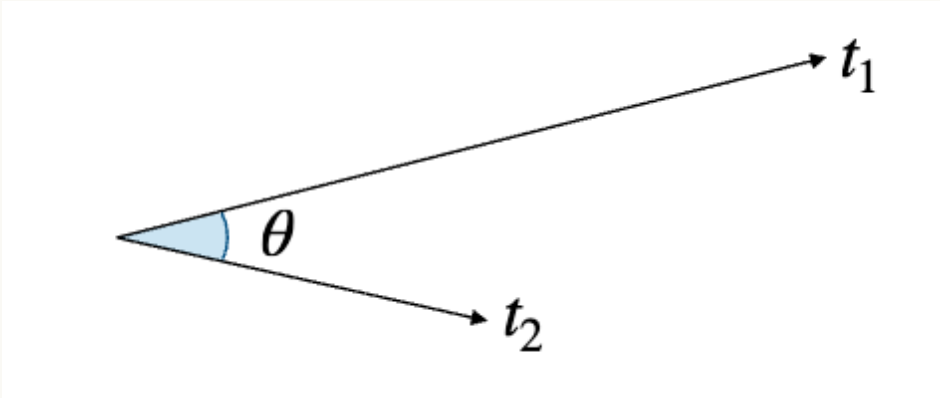
# Part I — Similarity

## How similar are two vectors?

Now that words and documents are vectors (lessons 1–2), "how similar are they?" becomes a geometry question: how *aligned* are their arrows? Two embeddings pointing the same direction are similar **regardless of length** — and length, in text embeddings, usually just reflects word frequency, not meaning.

> That's the whole idea behind **cosine similarity**: compare **angle**, not distance.

## The idea, in a picture




Two embeddings as arrows from a common origin. Everything cosine similarity measures is the **angle** $\theta$ — the two arrow *lengths* get divided out.

+1 = same direction (synonyms) · 0 = perpendicular (unrelated) · –1 = opposite (antonyms)

## The math, step by step

$$\text{cos-sim}(a, b) = \frac{a^\top b}{\|a\|\,\|b\|} = \cos\theta_{ab} \in [-1, 1] \tag{1}$$

1. $a^\top b = \sum_i a_i b_i$ — the **dot product**: large and positive when $a, b$ point the same way, negative when they point opposite ways, near zero when they're perpendicular.
2. $\|a\|\,\|b\|$ — dividing by both lengths **cancels out magnitude**, leaving only direction.
3. The ratio is exactly $\cos\theta_{ab}$, the cosine of the angle between $a$ and $b$ — which is why the result always lands in $[-1, 1]$. Here $\|\cdot\|$ is the ordinary (Euclidean) length of a vector, $\|a\| = \sqrt{\sum_i a_i^2}$.

## When distance and angle disagree (1/2)

Three 2-D toy embeddings: `lantern` $a = (2, 2)$, a frequent near-synonym `lamp` $b = (8, 8)$, and an unrelated word `tax` $c = (3, -1)$.

| Comparison | Euclidean distance | Cosine similarity |
|---|---|---|
| `lantern` vs. `tax` | $\|a - c\| = \sqrt{10} \approx 3.16$ | $\cos\theta_{ac} \approx 0.45$ |
| `lantern` vs. `lamp` | $\|a - b\| = 6\sqrt{2} \approx 8.49$ | $\cos\theta_{ab} = 1$ (exact — $b = 4a$) |

**Euclidean picks `tax`** as the nearer neighbor. **Cosine picks `lamp`**. They disagree.

## When distance and angle disagree (2/2)

The disagreement is pure **magnitude**: `lamp`'s length — in real embeddings, driven largely by word frequency, not meaning [8][9] — inflates its distance without changing its direction at all ($b = 4a$: same direction, just longer).

> Normalize all three vectors to unit length and the conflict **dissolves** — $a$ and $b$ literally coincide.

On **unit-normalized** vectors, cosine and Euclidean rank identically:

$$\|a - b\|^2 = 2 - 2\cos\theta_{ab} \tag{2}$$

Ranking by cosine similarity = ranking by (negative) distance — exactly why retrieval systems normalize, then compare with plain dot products.

## In practice: just the dot product

Two situations where only the **raw** $a^\top b$ gets used — no division:

1. **Retrieval.** If you already normalized your vectors, there's nothing left to divide by — the dot product **already is** the cosine similarity. Normalize once, then every comparison afterward is just a dot product.
2. **Training.** Vectors are usually left **unnormalized** on purpose: the vector's own magnitude carries information, and normalizing it away would throw that information out.

> **Rule of thumb:** normalize when you want a fair ranking. Leave vectors raw when their size itself means something.

## Cosine similarity: strengths & limitations

- ✅ Ignores magnitude by construction — exactly right when length reflects frequency, not meaning (the usual case for text embeddings).
- ✅ Bounded, interpretable range $[-1, 1]$; agrees with Euclidean distance once vectors are normalized (eq. 2).
- ⚠️ It's a **similarity**, not a distance — $1 - \cos\theta$ ("cosine distance") is the corresponding dissimilarity, and even that isn't a true metric.
- ⚠️ High cosine means "similar *contexts*," not "synonyms" — antonyms like `hot` / `cold` often score *high* because they appear in the same slots.

> ⚠️ **Misconception**: "Euclidean is always wrong for embeddings." It's fine *after* normalization (eq. 2) — the caution is about comparing raw, differently-scaled vectors directly.

# Part II — Dimension Reduction

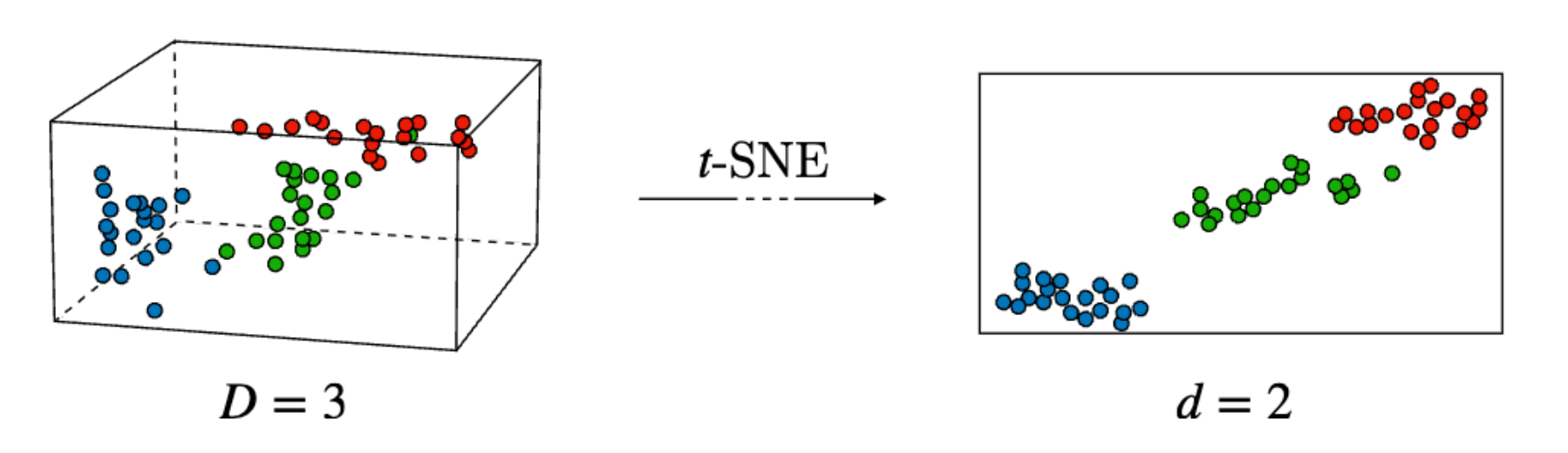
## Seeing a cloud you can't eyeball

Embeddings live in hundreds of dimensions — impossible to look at directly. To *see* them, we squash them down to 2-D.

The method that dominates in practice is **t-SNE**. It cares about one thing only: *neighborhoods*. It rearranges the points so that whoever was close in high-D stays close in 2-D — cheerfully distorting global distances to make **clusters pop**.

> That single design choice — preserve **local** structure, sacrifice **global** structure — explains everything t-SNE does well and every way it misleads you.

# The idea, in a picture



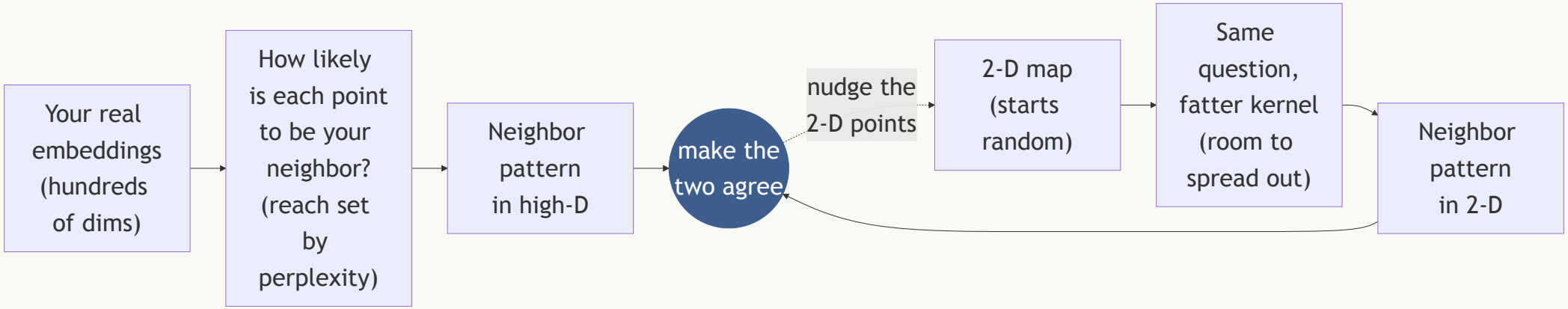

$D = 3$ becomes $d = 2$, and the three groups that overlapped inside the box come apart cleanly.

## t-SNE: the idea

t-SNE asks one narrow, local question: **for every point, who are its neighbors — and can the 2-D map keep them close?**

It never looks at variance or at any global axis. It looks at each point's local neighborhood — in both high-D and low-D — and tries to make the two **match**. The whole algorithm is that sentence, made precise in three steps.

# The pipeline, in a picture




Both sides answer the same question — *who is whose neighbor?* — and the map keeps moving until the two answers line up.

## Step 1 — turn distance into "are you my neighbor?"

Every point looks at every other point and converts the raw distance into a **probability of being its neighbor**: very close → high, far away → essentially zero.

Two consequences worth holding on to:

- The result is a **soft** neighborhood, not a hard cutoff — nothing is simply "in" or "out."
- Each point gets its **own reach**, adapted to how crowded its region is: tight in a dense cluster, wider out in sparse territory. That reach is what **perplexity** controls.

## The kernel is a choice, not a law

The function that converts distance into "neighborliness" is called a **kernel**. t-SNE's standard choice is a **Gaussian** — but nothing forces that.

> The source is explicit about this: t-SNE uses a Gaussian kernel, **but you could technically pick another function** to quantify similarity between observations — **cosine similarity**, for instance.

Which makes sense given Part I: cosine is *already* a similarity measure. Swapping the kernel changes how fast "neighborliness" falls off with distance, and therefore what the final map emphasizes.

## Step 2 — the 2-D map plays the same game

The flat map does exactly the same thing: for every pair of points, how likely is one to be the other's neighbor? Same question.

But it uses a **different, "fatter" kernel: the Student-*t* distribution** (instead of a Gaussian) — its tails decay much more slowly with distance.

> That's not a detail — it fixes a real problem (next slide), and it's where the **"t"** in **t**-SNE comes from.

## Why the flat map needs more room

High-dimensional space simply has more room than two dimensions. A point can have many neighbors spread around it in high-D; in 2-D, there is not enough space for all of them to remain at a similar distance.

> If the map preserved every original distance exactly, many of a point's neighbors would end up placed on top of one another.

The fatter kernel compensates for this: it allows moderately distant points to be placed farther apart on the map than the original distance would suggest. This effectively expands the available space in two dimensions, which is why t-SNE maps typically show clusters with clear separation between them.

## Step 3 — make the two pictures agree

Now t-SNE has two sets of neighbor-probabilities: one from the real embeddings, one from the 2-D map. It **slides the map points around** until the two agree as closely as possible.

The key is that "agreement" is judged **asymmetrically**:

- Pulling apart two points that really *were* neighbors → **heavily** penalized.
- Placing two unrelated points near each other → **barely** penalized.

> That lopsided penalty is exactly why t-SNE protects **local** structure and is careless with **global** structure — the single most important fact for reading its output honestly.

## Perplexity — the knob you'll actually turn

Read perplexity as **"how many neighbors each point should pay attention to."**

- **Low** (≈5) → each point looks only at its closest few → many small, tight islands; fine local detail, fragmented big picture.
- **High** (≈50) → each point averages over a wider circle → smoother, more consolidated layout; detail washes out.

You set **one** value for the whole run, and t-SNE adapts each point's reach to hit it — automatically tighter in dense regions, wider in sparse ones. Typical range: **5–50** (mini-lab D uses 30).

> ⚠️ There is no "correct" perplexity. Run **two or three** values and report which one you used — a map tuned until the clusters look nice is self-deception, not evidence.

## The second knob: how long it runs

t-SNE reaches its layout by **moving the points repeatedly**, so the number of iterations matters:

- **Too few** → the map hasn't settled; the shapes you see are partly leftover randomness from the starting positions.
- **Enough** → the layout stabilizes and stops changing much.
- **More beyond that** → mostly just costs time.

> If two runs of the same data look very different, suspect **too few iterations** (or a different random start) before you conclude anything about the data.

## t-SNE: strengths & limitations

- ✅ Reveals nonlinear cluster structure that a flat linear projection can hide.
- ✅ Two interpretable knobs — perplexity and iterations — with predictable effects.
- ⚠️ Distances **between** clusters are not meaningful; neither are cluster **sizes** — only "who is near whom" *within* a neighborhood.
- ⚠️ **Stochastic** (random init) — different runs/perplexities give different-looking maps. It's a **visualization**, not a feature extractor.
- ⚠️ $O(n^2)$ naively — run it on a **sample**, not a full corpus.

> ⚠️ **Misconception**: "well-separated clusters in a t-SNE plot mean those groups are far apart." The algorithm never promised that — it only promised that *neighbors stayed neighbors* (step 3). Between-cluster distance is an artifact of the layout.

## A closing note: PCA, the linear alternative

**PCA** takes a different approach: it looks for the **direction along which the data is most spread out**, and projects every point onto it.

> Picture rotating the cloud of points and looking at its shadow on a line. PCA picks the angle where that shadow is **widest** — the direction that captures the most variability. That direction is the first principal component.

If a second dimension is needed, PCA picks the next-best direction: the one that captures as much of the *remaining* spread as possible, at a right angle to the first.

## Which one, and when

| | PCA | t-SNE |
|---|---|---|
| Map | linear projection | nonlinear rearrangement |
| Preserves | **global** variance, large distances | **local** neighborhoods |
| Repeatable? | deterministic | stochastic (seed-dependent) |
| Cost | one eigendecomposition | iterative, $O(n^2)$ naively |
| Use it to | compress features, denoise, preprocess | **look** at cluster structure |

They answer different questions, so they are **complementary, not interchangeable** — and they compose: running PCA down to ~50 dimensions first is the standard way to make t-SNE fast and stable (it's `sklearn`'s `init="pca"`, which mini-lab C uses).

## → To the in-class lab

By now you have everything mini-lab C needs. Cosine nearest neighbors, `king − man + woman ≈ queen`-style analogies on pretrained GloVe, and a t-SNE map of ~150 words are all **handed to you**. Your job is to interrogate them:

- **E5** — build a similar/unrelated word-pair benchmark **that could fail**, and measure how sharply each space separates the two. A neighbor list always returns five words; the *separation* is the actual evidence.
- **E6** — measure the analogy trick's **real hit rate** across several relation types, then read the *wrong* answers: they fail systematically, not randomly.

# Part III — Fast Retrieval

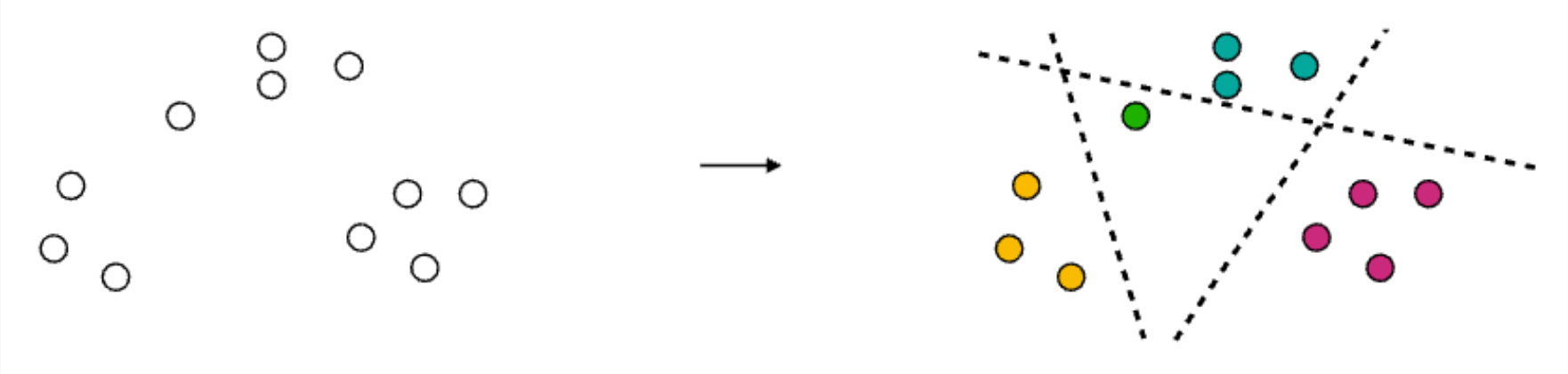
## Finding a needle in 100 million vectors

"Find the 10 most similar embeddings" is trivial for 1,000 vectors — just compare them all. It's hopeless for 100 million: a brute-force scan per query doesn't scale.

> **Locality-Sensitive Hashing (LSH)** cheats: design a hash that gives *similar* vectors the *same* bucket **on purpose**, then only compares within the query's bucket.

You trade exactness for speed — an **approximate** nearest neighbor that's good enough, and enormously faster.

## The idea, in a picture



Random hyperplanes carve the space into regions; **each region is one hash-table bucket**, and the color a point ends up with *is* its bucket.

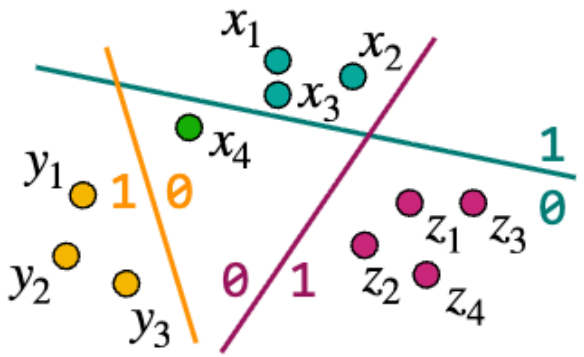
## Hashing with random hyperplanes

$$h_k(x) = \mathbf{1}\left[r_k^\top x \geq 0\right] \in \{0,1\}, \qquad \text{code}(x) = \big(h_1(x), \ldots, h_K(x)\big) \tag{3}$$

1. Draw $K$ random vectors $r_1, \ldots, r_K$ — normals to $K$ random hyperplanes through the origin.
2. Each hyperplane contributes **one bit**: which side of it does $x$ fall on?
3. The $K$-bit code names a **region** of space; all vectors in that region land in the same hash-table bucket.

With $K \sim \log n$ hyperplanes there are $2^K$ buckets and $\approx n/2^K$ items each — search becomes $O(\log n)$.

# Made concrete: $K = 3$ hyperplanes, 8 buckets




000 ⟶ $x_4$
001 ⟶ $z_1, z_2, z_3, z_4$
010 ⟶ $y_1, y_2, y_3$
011 ⟶ Ø
100 ⟶ $x_1, x_2, x_3$
101 ⟶ Ø
110 ⟶ Ø
111 ⟶ Ø

Each point's code reads **one bit per line**, in color order. $2^3 = 8$ codes exist — but only **4 are occupied**.

## What the worked example just showed us

- **Codes are read bitwise.** `z1` falls on the 0-side of the first line, 0 of the second, 1 of the third → code `001`, together with `z2, z3, z4`.
- **Buckets fill unevenly.** Four of the eight codes are **empty**. Real embeddings cluster by topic, so the tidy "$n/2^K$ items per bucket" estimate is optimistic — mini-lab D measures this on real data.
- **One unlucky hyperplane can split true neighbors.** `x4` sits right next to `x1, x2, x3`, but the first line passed *between* them: `x4` → `000`, the others → `100`. A genuine near neighbor now lives in a different bucket and will never be compared.

> That last failure is exactly why LSH in practice uses **several independent hash tables** and combines their results: a bad cut in one table is unlikely to repeat in another, so a true neighbor lost in one table can still turn up in the next.

## Where LSH fits

LSH is one **Approximate Nearest Neighbor (ANN)** method (others: KD-trees, and modern graph indexes like HNSW). ANN shines when:

- queries are **real-time** (recommendation, search),
- approximation is **acceptable**,
- or you need **near-duplicate detection** at scale.

> That's exactly the retrieval step inside a **RAG (retrieval-augmented generation)** pipeline — one of the most common real-world uses of everything in this section.

## LSH: strengths & limitations

- ✅ Turns an $O(n)$ brute-force scan into an $O(\log n)$ bucket lookup plus a tiny local comparison.
- ✅ Two knobs, one trade-off: $K$ (bucket precision, fewer false candidates) against $L$ (number of tables, recall).
- ⚠️ Recall $<$ 100% by design — you get a *likely* nearest neighbor, not the guaranteed exact one.
- ⚠️ Real embeddings cluster (by topic, language, ...), so buckets fill unevenly — plan for it, don't expect a clean $n/2^K$.

⚠️ **Misconception**: "More hyperplanes is always better." More bits → finer buckets → fewer false candidates, but also more true neighbors split apart (lower recall). You trade $K$ against $L$, you don't just maximize $K$.

## → To the lab

In-class **mini-lab D** hands you **three document encoders, already built and trained** (AG News) — mean-pooling (**Lesson 2**, eq. 5), a vanilla RNN, and an LSTM — scored by **retrieval precision@10**. You don't implement them: you **interrogate** them.

- **E7 — does word order actually matter?** Shuffle every document's tokens, re-score all three.
- **E8 — does the RNN↔LSTM gap grow with length?** Sweep sequence length; put **Lesson 2**'s eq. (10) on trial.
- **E9 — is precision@10 measuring relevance?** Audit the metric by hand.
- **E10 — the LSH speed/recall trade-off.** Map it, then pick an operating point and defend it.

**E8 is where Lesson 2 goes on trial:** eq. (9)/(10)'s vanishing gradient versus eq. (13)'s additive cell path, turned into a measured number.

## Recap

- **Cosine similarity** compares **direction, not length** — that's what makes it robust to frequency artifacts. On normalized vectors it ranks identically to Euclidean (eq. 2).
- **t-SNE** turns distances into **neighborhood probabilities** and minimizes their KL divergence. A fat-tailed Student-t kernel solves the **crowding problem**; **perplexity** sets "how many neighbors each point considers."
- **Read a t-SNE map honestly**: only *who is near whom* means anything — never between-cluster distances or cluster sizes.
- **PCA** is the linear, deterministic, global counterpart — a different question, and the standard preprocessing step before t-SNE.
- **LSH** buckets similar vectors **on purpose**: collision probability is a clean function of the angle, sharpened by $K$ and recovered by $L$.

## What's next

You now have the full pipeline: text → tokens → token vectors → document vectors → compared, visualized, and searched. **Lesson 4** puts those pieces to work: an LSTM encoder–decoder that *generates* — a sequence-to-sequence translator, built from the same attention idea Lesson 2 introduced as weighted pooling.

Beyond that, **Module 2 — Transformers** takes that same seed and makes it the *entire* architecture — self-attention, computed for every position in parallel, with no recurrence at all.

Curious now? van der Maaten & Hinton [2] is worth reading in full — the original t-SNE paper is unusually readable — and Charikar [7] gives the complete random-hyperplane LSH analysis in a few clean pages.

# References & further reading (1/3)

## Primary source


- [1] A. Amidi and S. Amidi, *Super Study Guide: Transformers and Large Language Models*, Part 2 "Embeddings," 2024 — primary source for this lesson.


## Word-vector geometry (Part I)


- [8] A. M. J. Schakel and B. J. Wilson, "Measuring Word Significance using Distributed Representations of Words," *arXiv:1508.02297*, 2015 — vector **length** as word significance; **semantic** content lives in **direction**.
- [9] B. J. Wilson and A. M. J. Schakel, "Controlled Experiments for Word Embeddings," *arXiv:1510.02675*, 2015 — word2vec vector **length grows ~linearly with word frequency**, direction held fixed by design.

# References & further reading (2/3)

## Dimension reduction


- [2] L. van der Maaten and G. Hinton, "Visualizing Data using t-SNE," *Journal of Machine Learning Research*, vol. 9, 2008.
- [3] G. Hinton and S. Roweis, "Stochastic Neighbor Embedding," in *Proc. NeurIPS (NIPS 15)*, 2002, pp. 833–840.
- [4] I. T. Jolliffe, *Principal Component Analysis*, 2nd ed., Springer, 2002.
- [5] H. Hotelling, "Analysis of a Complex of Statistical Variables into Principal Components," *Journal of Educational Psychology*, 1933.

# References & further reading (3/3)

## Fast retrieval / LSH


- [6] P. Indyk and R. Motwani, "Approximate Nearest Neighbors: Towards Removing the Curse of Dimensionality," in *Proc. STOC*, 1998.
- [7] M. Charikar, "Similarity Estimation Techniques from Rounding Algorithms," in *Proc. STOC*, 2002, pp. 380–388.


## Worth reading in full

van der Maaten & Hinton [2] — the original t-SNE paper is unusually readable. Charikar [7] gives the complete random-hyperplane analysis in a few clean pages.